

Logistic Regression: Loss and Cost Functions

Syed Hamed Raza

University of Management and Technology (UMT), Lahore

Course: Deep Learning and Neural Network

Course Code: CS4152

November 6, 2025

Recap: Logistic Regression Model

- Prediction: $\hat{y} = \sigma(W^T x + b)$ with $\sigma(z) = \frac{1}{1+e^{-z}}$.
- Parameters to learn: $W \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$.
- Need an objective to measure performance on the training set.

Training Objective

Given training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^m$ with $y^{(i)} \in \{0, 1\}$:

$$\hat{y}^{(i)} = \sigma(W^T x^{(i)} + b)$$

$$z^{(i)} = W^T x^{(i)} + b$$

Choose W, b so that predictions $\hat{y}^{(i)}$ are close to labels $y^{(i)}$.

Why Not Squared Error?

- Squared error: $L_{SE}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$.
- For logistic regression, this leads to a **non-convex** objective.
- Gradient descent can get stuck in local minima / saddle points.
- Use **logistic (cross-entropy) loss** for a convex optimization landscape.

Logistic Loss (Binary Cross-Entropy)

Loss for a single example:

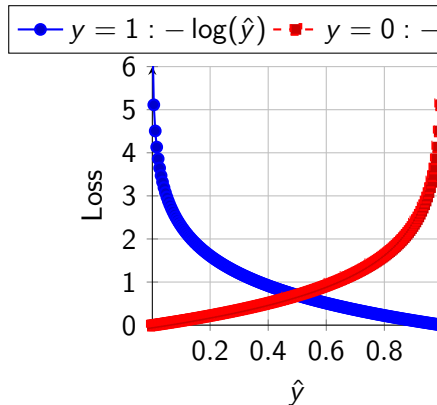
$$L(\hat{y}, y) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \right].$$

Intuition:

- If $y = 1$: $L = -\log(\hat{y}) \Rightarrow$ minimize by making $\hat{y}$ large.
- If $y = 0$: $L = -\log(1 - \hat{y}) \Rightarrow$ minimize by making $\hat{y}$ small.

Visual Intuition for the Loss

- Steep penalty for confident wrong predictions.
- Smooth gradients for stable learning.



Cost Function (Dataset-Level)

Average loss over the training set:

$$J(W, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$$

Goal: Find W, b that minimize $J(W, b)$.

Gradient Descent (Preview)

Parameter updates:

$$W \leftarrow W - \alpha \frac{\partial J}{\partial W},$$
$$b \leftarrow b - \alpha \frac{\partial J}{\partial b},$$

where α is the learning rate.

- Cross-entropy yields well-behaved gradients for logistic regression.
- Vectorized implementations compute these updates efficiently.

Terminology Recap

Term	Definition	Scope
Loss $L(\hat{y}, y)$	Error for a single example	One sample
Cost $J(W, b)$	Mean of losses over dataset	Entire training set

Summary

Logistic Regression cost function

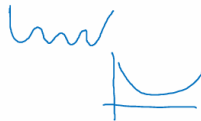
$$\rightarrow \hat{y}^{(i)} = \sigma(\underline{w^T x^{(i)} + b}), \text{ where } \underline{\sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}} \quad z^{(i)} = w^T x^{(i)} + b$$

Given $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, want $\hat{y}^{(i)} \approx y^{(i)}$.

$x^{(i)}$
 $y^{(i)}$
 $z^{(i)}$ i -th example.

Loss (error) function: $\underline{\ell(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2}$

$$\boxed{\ell(\hat{y}, y)} = - \left[\underline{y \log \hat{y}} + \underline{(1-y) \log(1-\hat{y})} \right] \leftarrow$$



If $y=1$: $\ell(\hat{y}, y) = -\log \hat{y} \leftarrow$ want $\log \hat{y}$ large, want $\hat{y}$ large.

If $y=0$: $\ell(\hat{y}, y) = -\log(1-\hat{y}) \leftarrow$ want $\log(1-\hat{y})$ large ... want $\hat{y}$ small

$$\boxed{\text{Cost}} \text{ function: } J(\underline{w}, \underline{b}) = \frac{1}{m} \sum_{i=1}^m \ell(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-\hat{y}^{(i)}) \right]$$

Andrew Ng

Summary and Next Steps

- Binary cross-entropy: convex, stable, probability-aware objective.
- Cost function $J(W, b)$ averages per-example losses.
- Next: view logistic regression as a **tiny neural network** and derive gradients.